

LEGO Resale Intelligence.

Cómo construir una herramienta de e-commerce intelligence sobre LEGO descatalogado, paso a paso, partiendo desde cero código — con criterio de producto, lógica de negocio real y resultado visualmente premium.

PDF-READY · PORTFOLIO-GRADE

Perfil objetivo	Tiempo estimado	Stack
Producto + Datos · Ambicioso junior	12 semanas · 5–8 h/semana	Pythron · Streamlit · SQLite · LLM API



Índice

A	Resumen ejecutivo	03
B	Visión del producto	05
C	Alcance de la v1 — qué entra y qué no	07
D	Stack recomendado	09
E	Arquitectura del sistema	11
F	Modelo de datos	13
G	Lógica de negocio	15
H	Diseño visual y UX/UI	17
I	Roadmap paso a paso	20
J	Plan de construcción con Codex	23
K	Entregables finales	27
L	Cómo contar el proyecto	28
M	Riesgos y errores a evitar	30

▲ HONESTIDAD BRUTAL ANTES DE EMPEZAR

Esta guía está calibrada para alguien que **nunca ha programado** y dispone de **5–8 h/semana**. La v1 propuesta es deliberadamente conservadora: una sola fuente de datos, un solo flujo, una sola pantalla bien hecha.

Si en algún punto sientes que vas demasiado rápido y no entiendes lo que Codex genera, **frena**. Un proyecto que no puedes explicar en una entrevista es peor que no tener proyecto.

Resumen ejecutivo

Una herramienta de e-commerce intelligence que detecta oportunidades de compra de LEGO descatalogado en marketplaces de segunda mano, estima márgenes reales y prioriza decisiones — pensada como pieza de portfolio defendible, no como bot de arbitraje.

12SEMANAS DE
EJECUCIÓN**1**MARKETPLACE
FUENTE**~150**LISTINGS
ANALIZADOS**5**

VISTAS PREMIUM UI

El problema

El mercado de LEGO descatalogado en plataformas C2C (Wallapop, Vinted, eBay) está fragmentado y opaco. Los precios oscilan brutalmente para el mismo set por desconocimiento del vendedor, condiciones del producto, y falta de comparables claros. Comprar bien requiere tiempo, criterio y memoria de cientos de referencias — tres recursos escasos.

La solución

Una herramienta que ingiere listings, los normaliza, los enriquece con datos de referencia (set ID, año de retirada, precio de mercado), calcula un **score de oportunidad** ponderado y permite tomar decisiones de compra fundamentadas en segundos en lugar de horas.

Por qué LEGO descatalogado

- **Mercado real y medible:** el efecto retirada sobre precios LEGO está documentado (rentabilidad histórica >7% anual en sets seleccionados según estudios académicos como el de Victoria Dobrynskaya).
- **Identificación inequívoca:** cada set tiene un ID numérico único — facilita el matching y el modelo de datos.
- **Datos de referencia accesibles:** Brickset, BrickLink y Rebrickable ofrecen catálogos consultables.
- **Nicho con personalidad:** en una entrevista, decir "construí inteligencia de pricing sobre LEGO retirado" es mil veces más memorable que "hice un dashboard de e-commerce genérico".

Por qué este proyecto sirve para CV/portfolio

DEMUESTRA PRODUCTO

Criterio de scope

Saber qué dejar fuera importa más que saber qué meter. Una v1 acotada y defendible vale más que una demo hinchada.

DEMUESTRA DATOS

Lógica de negocio real

Pricing, márgenes netos, scoring ponderado. No un dashboard hueco — decisiones cuantitativas reales.

DEMUESTRA EJECUCIÓN

End-to-end funcional

Captura → normalización → análisis → visualización.
El flujo completo, no fragmentos sueltos.

DEMUESTRA IA APLICADA

Uso responsable de LLMs

Codex como copiloto, LLM para extracción estructurada — IA con propósito, no por moda.

Resultado final esperado

Una aplicación web local funcional + dashboard visual + repo público con README impecable + case study en PDF + post de LinkedIn + 3 bullets afilados para CV. Todo coherente, todo defendible, todo enseñable.

Visión del producto

Antes de escribir una línea de código, hay que tener cristalino qué hace el sistema, qué no hace, y qué valor entrega. Esta sección es la brújula del proyecto.

Qué hace el sistema (en una frase)

★ PITCH DE UNA FRASE

"Convierte listings dispersos de LEGO descatalogado en decisiones de compra priorizadas, con margen estimado y nivel de riesgo."

Qué hará exactamente la v1

- 1. **Capturar** listings de una sola fuente (semi-manual + parser).
- 2. **Identificar** el set LEGO (ID, nombre, año, precio retail original).
- 3. **Estimar** precio de mercado actual contra una tabla de comparables.
- 4. **Calcular** margen bruto y margen neto (con comisiones y envío).
- 5. **Puntuar** la oportunidad con un score 0–100.
- 6. **Mostrar** el ranking de oportunidades en un dashboard navegable.
- 7. **Permitir** marcar listings como "comprado", "descartado" o "vigilando".

Qué NO hará (explícito y consciente)

Función excluida	Razón
Compra automática	Ético, legal y técnicamente complicado. Devalúa el proyecto.
Scraping masivo no autorizado	ToS, riesgo legal, ruido en la narrativa. Mejor parser controlado o input semi-manual.
Multi-marketplace	Triplika el scope sin triplicar el aprendizaje. Una fuente bien hecha > tres mediocres.
Recomendador con ML pesado	No aporta valor real en v1, sí aporta riesgo de no entender el modelo.

Función excluida	Razón
App móvil / sistema multiusuario	Fuera de scope. Streamlit local es suficiente para portfolio.
Publicación / reventa automática	Otro producto. Si el v1 funciona, se evalúa después.

Inputs y outputs

Inputs

- URL del listing (manual o pegado)
- Foto del listing (opcional, para Vision)
- Catálogo de referencia (CSV con sets retirados)
- Tabla de comparables (CSV inicial)
- Parámetros de coste (envío, comisión)

Outputs

- Ficha estructurada del listing
- Precio razonable estimado (rango)
- Margen bruto y margen neto en €
- Score de oportunidad 0–100
- Etiqueta visual:  /  / 
- Flags de riesgo

Valor entregado

Para un comprador-revendedor: **reducir el tiempo de evaluación de un listing de 15 minutos a 15 segundos**. Para tu portfolio: **demostrar que sabes traducir un problema real de mercado en producto, datos y código funcional**.

Alcance de la v1

Una v1 que se pueda terminar en 12 semanas a 5–8 h/semana. Todo lo que no entre aquí es ruido — anótalo en un "post-v1.txt" y olvídalo hasta que la v1 esté enseñable.

Matriz IN / OUT

Categoría	IN — v1	OUT — post-v1
Fuentes	1 marketplace (recomendado: eBay por API pública o Wallapop por parsing controlado)	Vinted, Milanuncios, Catawiki, Facebook Marketplace
Captura	Pegar URL/HTML manual + parser local	Crawler programado, alertas en tiempo real
Extracción	LLM de un proveedor (OpenAI o Anthropic) sobre texto	Vision LLM sobre fotos, OCR de cajas
Catálogo	CSV estático con ~200 sets retirados top	API Brickset/Rebrickable con sync automático
Pricing	Tabla manual de comparables (€ por set)	Histórico, tendencias, predicción
Scoring	Fórmula ponderada con pesos transparentes	Modelo ML, A/B testing de pesos
UI	Streamlit · 5 vistas · diseño premium	Next.js, móvil, multiusuario
Persistencia	SQLite local	Postgres en la nube, backups
Despliegue	Local + opcional Streamlit Community Cloud para demo	Docker, CI/CD, monitorización
Usuarios	Tú (single-user)	Auth, roles, equipos

Qué se simula al principio (y por qué está bien)

NOTE · SIMULACIÓN INTELIGENTE

En las primeras semanas, el "catálogo de comparables" será un CSV que has rellenado tú a mano con 30–50 sets más buscados. Esto NO es trampa — es la forma correcta de validar que la lógica del sistema

funciona antes de invertir tiempo en automatizar la fuente de datos. En entrevistas se cuenta así: *"empecé con datos curados manualmente para validar el modelo de negocio antes de automatizar la captura"*.

Qué es manual / qué es automático en la v1

Paso	Modo v1	Por qué
Encontrar listings	Manual (pegas URL)	Evita scraping masivo y ToS
Parsear listing	Automático (LLM)	Donde la IA aporta valor real
Identificar set	Automático (matching)	Lógica determinista clara
Comparables	CSV curado por ti	Validas modelo antes de automatizar
Score	Automático	Fórmula transparente
Decisión final	Manual humana	Sistema ayuda, no decide por ti

⚠ ERROR QUE DESTRUIRÍA EL SCOPE

Querer scrapear "todo Wallapop en tiempo real" la semana 2. No lo hagas. Vas a perder 3 semanas peleándote con anti-bot, captchas y bloqueos, sin haber escrito una sola línea de lógica de negocio. Es el error nº1 en proyectos de scraping de junior.

Stack recomendado

Stack mínimo viable para alguien que parte de cero código. Cada elección está optimizada para: tiempo de aprendizaje, capacidad de defender en entrevista, y resultado visual.

Stack final propuesto

Capa	Elección	Por qué (defensa en entrevista)
Lenguaje	Python 3.11+	Estándar absoluto en data/ML/scripting. Sintaxis legible para principiantes.
UI	Streamlit	Permite tener una UI funcional con <100 líneas. Codex la genera muy bien. Profesionalmente aceptable como proof-of-concept.
BBDD	SQLite (vía SQLAlchemy)	Cero setup. Un archivo .db. Migrar a Postgres después es trivial.
Parsing HTML	BeautifulSoup4 + requests	Estándar de facto. Documentación abundante.
LLM	API de Anthropic o OpenAI	Una elección. Empezar con la que ya tengas cuenta. Coste estimado v1: <5€.
Validación de datos	Pydantic	Estructura tus datos, evita errores tontos. Codex lo usa muy bien.
Visualización	Plotly (dentro de Streamlit)	Interactivo, profesional, exportable.
Entorno	uv o venv + requirements.txt	Reproducibile. uv es más moderno; venv es más estándar.
Editor	VS Code + extensión Python	Estándar. Codex se integra nativo.
Git	Git + GitHub público	Imprescindible para portfolio. Cada commit cuenta tu historia.

Capa	Elección	Por qué (defensa en entrevista)
Deploy demo	Streamlit Community Cloud (gratis)	1 click. URL pública para LinkedIn.

Por qué NO usar (todavía)

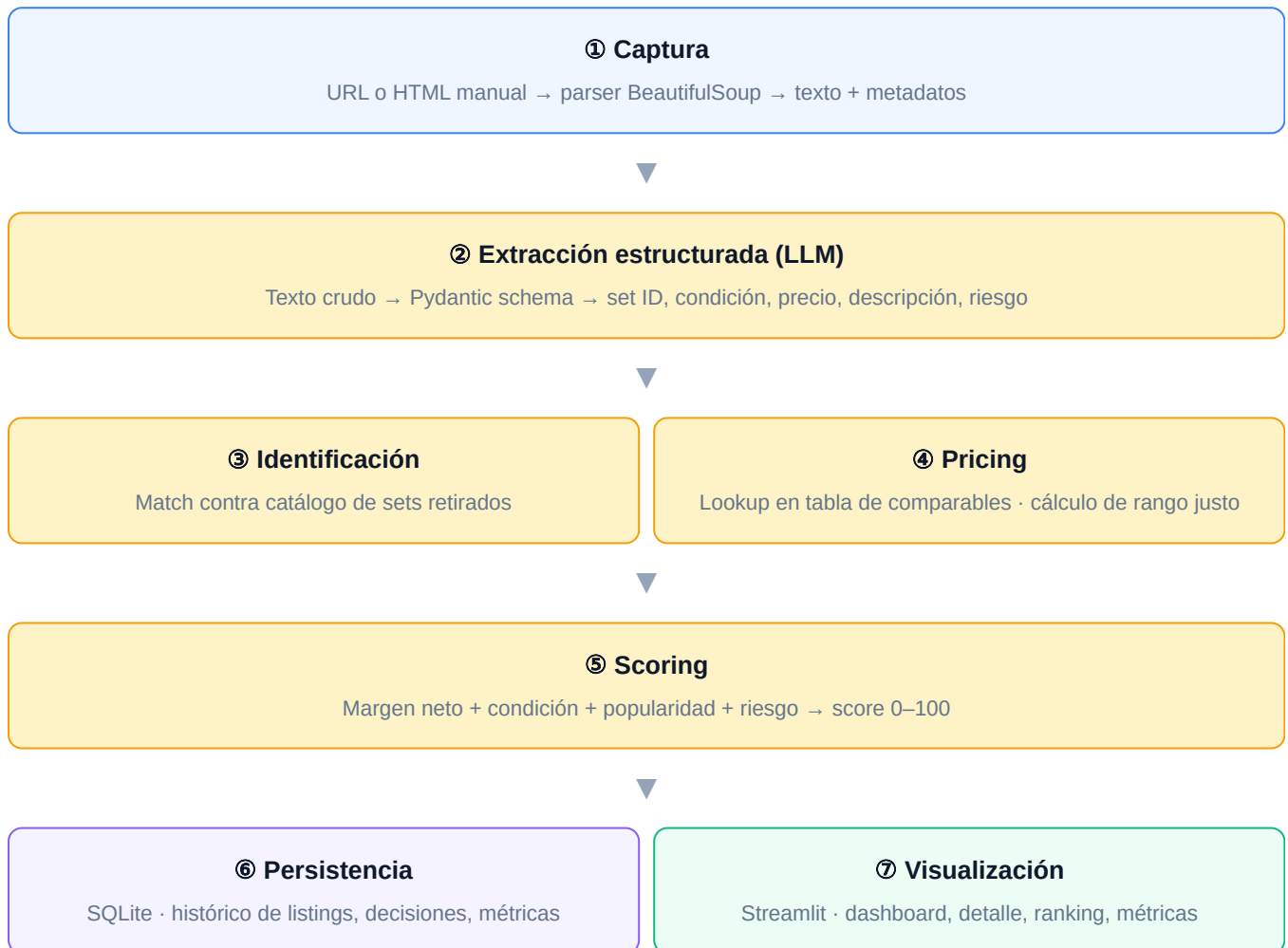
FastAPI / backend separado Sobreingeniería para v1. Streamlit hace ambas funciones. Lo añadirías post-v1 si separas backend de frontend.	Next.js / React Otro lenguaje, otra curva. Multiplica el tiempo x3. Solo tendría sentido si tu rol objetivo fuera frontend dev.
Postgres / pgvector No necesitas vectores en v1. SQLite migrable después.	Docker Cuando tengas la app funcionando lo añades en 1 hora. Antes es distracción.
Embeddings / KNN Tienes IDs únicos de set. Matching exacto resuelve el 95% de los casos.	Vision LLM Excelente feature post-v1 para diferenciar. En v1 no es ruta crítica.

Costes estimados (v1 completa)

0€ HOSTING	~5€ LLM API	0€ BBDD	~5€ TOTAL V1
---------------------------------	----------------------------------	------------------------------	-----------------------------------

Arquitectura del sistema

Arquitectura modular en 6 bloques. Cada módulo tiene una responsabilidad única, se prueba aislado y se sustituye sin romper el resto. Esto es lo que se espera ver en cualquier proyecto técnico serio.



Estructura de carpetas recomendada

```
lego-resale-intelligence/  
├── README.md  
├── requirements.txt  
├── .env.example      ← claves API (sin commitear claves reales)  
├── .gitignore  
├──  
├── data/  
│   ├── catalog.csv    ← sets retirados de referencia  
│   └── comparables.csv ← precios de mercado curados
```

```

├── db/
│   └── lri.db          ← SQLite (no se commitea)
├── src/
│   ├── __init__.py
│   ├── capture.py      ← módulo ①
│   ├── extract.py      ← módulo ② (llamada al LLM)
│   ├── identify.py     ← módulo ③
│   ├── pricing.py      ← módulo ④
│   ├── scoring.py      ← módulo ⑤
│   ├── models.py       ← Pydantic + SQLAlchemy
│   └── db.py           ← módulo ⑥ (CRUD)
├── app/
│   ├── main.py         ← Streamlit entry point
│   ├── pages/
│   │   ├── 1_📊_Dashboard.py
│   │   ├── 2_🔍_Oportunidades.py
│   │   ├── 3_🏠_Detalle.py
│   │   ├── 4_📈_Métricas.py
│   │   └── 5_⚙️_Operaciones.py
│   └── components/     ← gráficos, cards, helpers
├── tests/
│   ├── test_pricing.py
│   ├── test_scoring.py
│   └── test_extract.py
└── docs/
    ├── architecture.png
    ├── case_study.pdf
    └── screenshots/

```

★ RECOMENDACIÓN CLAVE

No empieces creando todas las carpetas. Empieza con un solo archivo `main.py` en la raíz, valida que la lógica funciona, y solo entonces refactoriza a esta estructura. Esto se llama "build to learn, refactor to scale" — y es lo que hace cualquier ingeniero senior.

Modelo de datos

El esquema mínimo necesario. Diseñado para ser autodocumentado, expandible y entendible por cualquier persona que lo abra por primera vez.

Tabla: `set_catalog`

Catálogo de referencia de sets retirados. Se carga desde CSV al inicializar la BBDD.

Campo	Tipo	Descripción
<code>set_id</code>	STRING (PK)	ID oficial LEGO, ej. "10221"
<code>name</code>	STRING	"Super Star Destroyer"
<code>theme</code>	STRING	"Star Wars"
<code>year_released</code>	INT	2011
<code>year_retired</code>	INT	2014
<code>retail_price_eur</code>	FLOAT	449.99
<code>pieces</code>	INT	3152
<code>popularity_score</code>	INT	1–10 (curado manualmente)

Tabla: `comparables`

Precios de mercado de referencia para sets sellados / usados.

Campo	Tipo	Descripción
<code>set_id</code>	STRING (FK)	→ <code>set_catalog</code>
<code>condition</code>	ENUM	SEALED · USED_COMPLETE · USED_INCOMPLETE
<code>price_min_eur</code>	FLOAT	Mínimo razonable
<code>price_avg_eur</code>	FLOAT	Media de mercado

Campo	Tipo	Descripción
<code>price_max_eur</code>	FLOAT	Máximo razonable
<code>last_updated</code>	DATE	Cuándo se actualizó
<code>source</code>	STRING	"manual" · "ebay_sold" · etc.

Tabla: `listings`

El corazón del sistema. Cada fila es un listing analizado.

Campo	Tipo	Descripción
<code>id</code>	INT (PK)	Autoincremental
<code>url</code>	STRING (UNIQUE)	URL del listing
<code>marketplace</code>	STRING	"ebay", "wallapop"...
<code>captured_at</code>	DATETIME	Cuándo lo capturaste
<code>raw_title</code>	STRING	Título original
<code>raw_description</code>	TEXT	Descripción completa
<code>asking_price_eur</code>	FLOAT	Lo que pide el vendedor
<code>shipping_eur</code>	FLOAT	Coste de envío
<code>set_id</code>	STRING (FK)	→ <code>set_catalog</code> (puede ser NULL si no se identifica)
<code>condition</code>	ENUM	SEALED / USED_COMPLETE / USED_INCOMPLETE / UNKNOWN
<code>fair_price_eur</code>	FLOAT	Precio razonable estimado
<code>gross_margin_eur</code>	FLOAT	<code>fair_price</code> – <code>asking_price</code>
<code>net_margin_eur</code>	FLOAT	Margen tras comisiones y envío
<code>opportunity_score</code>	INT	0–100

Campo	Tipo	Descripción
<code>risk_flags</code>	JSON	["foto_unica", "vendedor_nuevo", ...]
<code>status</code>	ENUM	NEW · WATCHING · BOUGHT · DISCARDED
<code>notes</code>	TEXT	Notas manuales

Tabla: `operations`

Histórico de decisiones (audit trail).

Campo	Tipo	Descripción
<code>id</code>	INT (PK)	Autoincremental
<code>listing_id</code>	INT (FK)	→ listings
<code>action</code>	ENUM	ANALYZED · MARKED_WATCHING · BOUGHT · DISCARDED
<code>timestamp</code>	DATETIME	Cuándo
<code>actual_price_paid_eur</code>	FLOAT	Si se compró
<code>reason</code>	TEXT	Por qué se descartó / compró

NOTE · ESTO IMPORTA

La tabla `operations` es la que diferencia un proyecto serio de una demo. Te permite calcular después **"acerté el X% de las veces"**, **"el score predice el margen real con un R² de Y"**. Esto es oro en una entrevista. No la omitas.

Lógica de negocio

El cerebro del sistema. Cada fórmula está diseñada para ser transparente, defendible y ajustable. Nada de cajas negras — todos los pesos son explícitos.

1. Precio razonable (fair_price)

Para un listing identificado correctamente con `set_id` y `condition`:

```
fair_price = comparables.price_avg_eur
fair_range = (comparables.price_min_eur, comparables.price_max_eur)
```

Si la condición es `UNKNOWN`, asumir `USED_COMPLETE` y aplicar penalización en el score.

2. Margen bruto

```
gross_margin_eur = fair_price - (asking_price + shipping)
gross_margin_pct = gross_margin_eur / asking_price * 100
```

3. Margen neto (con costes reales)

Aquí entra el realismo: revender tiene costes.

```
FEES_RATE = 0.10      # comisión de plataforma típica al vender
SHIPPING_OUT = 8.0    # envío al revender (estimado)
MARGIN_BUFFER = 0.05  # buffer por imprevistos

resale_revenue = fair_price * (1 - FEES_RATE)
total_cost = asking_price + shipping
net_margin_eur = resale_revenue - total_cost - SHIPPING_OUT
net_margin_pct = net_margin_eur / total_cost * 100
```

4. Score de oportunidad (0–100)

Combinación ponderada de 4 factores:

Factor	Peso	Cálculo
Margen neto %	50%	Normalizado: 0% margen → 0 pts · ≥40% margen → 100 pts · lineal entre medias
Popularidad del set	20%	<code>popularity_score</code> del catálogo · 1–10 → 0–100
Confianza en condición	20%	SEALED=100 · USED_COMPLETE=70 · USED_INCOMPLETE=30 · UNKNOWN=20
Penalización por riesgo	10%	100 – (10 × nº de <code>risk_flags</code>), mínimo 0

```
score = (
    margin_score * 0.50 +
    popularity_score * 0.20 +
    condition_confidence * 0.20 +
    risk_score * 0.10
)
```

5. Categorización visual

Oportunidad

Score ≥ 70 · margen neto > 25%

Vigilar

Score 50–69 · revisar manualmente

Descartar

Score < 50 · margen bajo o riesgo alto

6. Detección de riesgo (`risk_flags`)

El LLM se prompta para detectar señales de riesgo en la descripción:

- **foto_unica** — solo una foto disponible
- **caja_dañada** — menciones de "caja con golpes"
- **incompleto** — "faltan piezas", "no sé si está completo"
- **sin_instrucciones** — "sin manual"
- **vendedor_no_responde** — banderas de comportamiento del vendedor
- **precio_fuera_rango** — `asking_price` < `price_min` × 0.5 (sospechoso)
- **set_no_identificado** — sin `set_id` confirmado

⚠ VARIABLES QUE IMPORTAN MÁS DE LO QUE PARECE

- **Condición** es el factor más infravalorado por compradores junior. Un sealed vs used del mismo set pueden tener 3x diferencia de precio.
- **Popularidad** determina la liquidez. Un set raro con 30% margen es peor que uno popular con 25% — porque el popular se vende en 1 semana y el raro en 6 meses.
- **Riesgo** no se debe sumar como "1 flag = -10 puntos" sino que ciertos flags individuales (incompleto, sin instrucciones) deberían ser vetos: score automático < 30 pase lo que pase.

Diseño visual y UX/UI

Esta sección decide si el proyecto se percibe como serio o cutre. Una v1 con poca lógica pero diseño impecable se vende mejor que una v1 con lógica brillante y diseño feo. Tómalo en serio.

Filosofía de diseño

"Un dashboard analítico se mira durante horas. Cada decisión visual debe reducir fricción cognitiva, no añadirla."

Inspiración para mood board: **Linear · Vercel · Stripe Dashboard · Pitch · Arc Browser**. Lo opuesto de lo que queremos: paneles cargados estilo Bloomberg, dashboards de admin de Bootstrap genérico, Material UI plano.

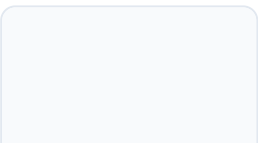
Paleta cromática



Ink
#0a0e27



LEGO Yellow
#fbbf24



Surface
#f8fafc



Signal Up
#10b981



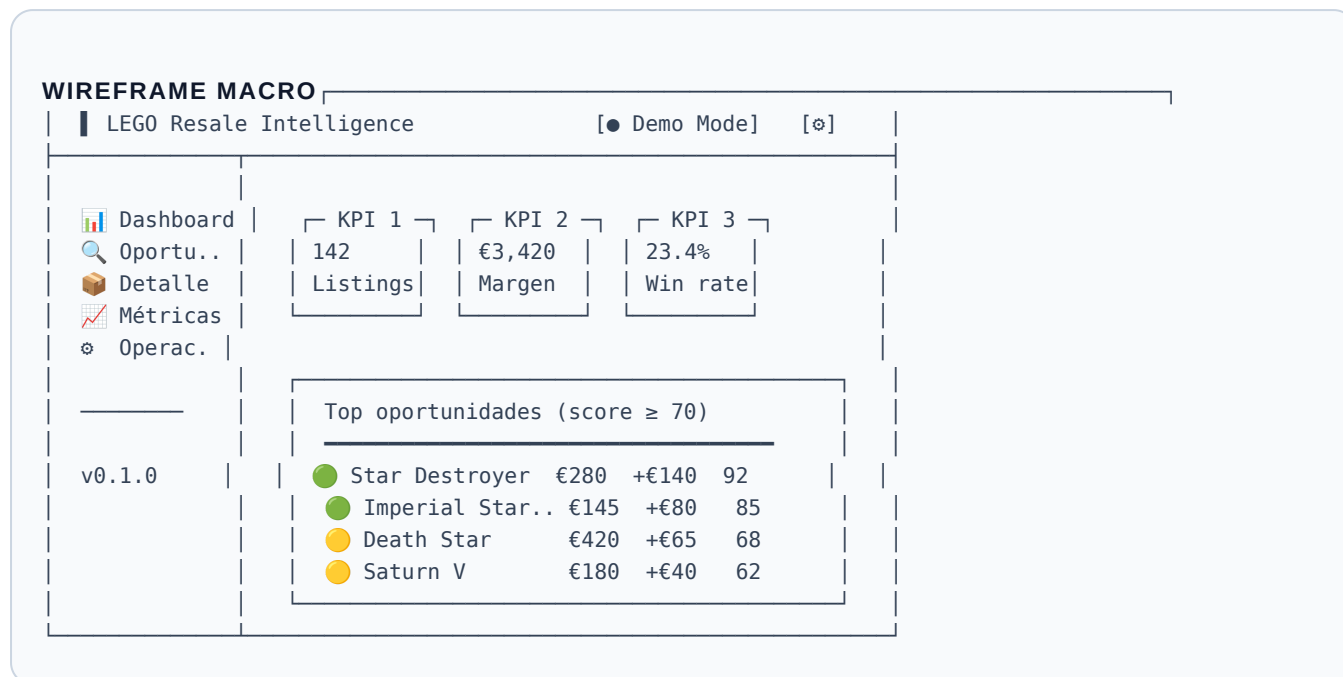
Signal Down
#ef4444

Regla: el amarillo solo para acentos críticos (CTAs, scores altos, valores destacados). Si lo usas en todo, deja de funcionar como acento.

Tipografía

Uso	Tipografía	Tamaños
Headlines & títulos	Inter (700/800)	32px / 24px / 18px
Body	Inter (400/500)	14px / 13px
Datos numéricos & métricas	JetBrains Mono o Inter (tabular-nums)	32px / 16px
Labels técnicos	Inter (600, uppercase, tracking)	10px / 11px

Layout general (Streamlit)



Pantallas mínimas — las 5 vistas v1

1. Dashboard principal

- 4 KPIs cabecera: total listings, oportunidades activas, margen total potencial, win rate histórico
- Tabla "Top 10 oportunidades" con score, margen, badge de condición
- Gráfico Plotly: distribución de scores
- Mini-card "Last analyzed" — último listing procesado

2. Vista de oportunidades (lista)

- Filtros laterales: theme, score min, condición, margen mín, status
- Cards en grid o tabla densa (toggle)
- Ordenable por: score, margen, fecha
- Quick actions: marcar como watching / discarded

3. Detalle de listing

- Header: imagen del set + nombre + set_id
- Comparativa visual: asking_price vs fair_range vs net_margin
- Breakdown del score con barras
- Risk flags como chips de colores
- Notas manuales editables
- Histórico de comparables del set

4. Vista comparables / catálogo

- Tabla del catálogo de sets con filtros
- Por set: rango de precios, popularidad, gráfico de evolución
- Fuente de datos visible (transparencia)

5. Métricas / Operaciones

- Histórico de decisiones
- "Predicción vs realidad" — para los listings comprados
- Funnel: analyzed → watching → bought
- Distribución de risk flags

Componentes esenciales

Score badge

Badge circular con número grande, color condicional, label "score". Tamaño consistente en toda la app.

Margin pill

Pill con €X (+Y%), color verde/rojo, formato financiero. Importante: signo siempre visible.

Condition chip

SEALED / USED · color codificado · icono pequeño · misma altura siempre.

KPI card

Valor grande tabular-nums + label uppercase pequeño + delta opcional. Sin iconos decorativos.

Listing card

Imagen 16:9 + título + set_id + score badge esquina + margen pill + estado footer.

Empty state

Cuando no hay listings: ilustración SVG simple + CTA grande "Pegar primer listing". Crítico para primera impresión.

Reglas de jerarquía visual

1. **Datos > chrome.** Quita bordes, sombras, gradientes innecesarios. El contenido respira.
2. **Una sola jerarquía de color.** Si todo grita, nada se oye. El amarillo solo para 1–2 elementos por pantalla.
3. **Espacio en blanco generoso.** Mínimo 24px de padding en cards, 16px entre componentes.
4. **Tipografía tabular para números.** `font-variant-numeric: tabular-nums`. Las columnas de números se alinean visualmente.
5. **Estados visibles.** Hover, loading, empty, error — los 4 estados básicos diseñados.

★ CÓMO CONSEGUIR UI PREMIUM EN STREAMLIT

Streamlit puede verse genérico por defecto. Tres trucos: **(1)** usa `st.set_page_config()` con tema custom; **(2)** inyecta CSS propio vía `st.markdown('<style>...</style>', unsafe_allow_html=True)` ; **(3)** usa componentes de la librería `streamlit-elements` o renderiza HTML/CSS para los componentes críticos (cards, badges). Esto te permite que la UI parezca un dashboard de Linear, no un demo de Streamlit.

Roadmap paso a paso

12 semanas · 5 fases · cada fase con un entregable concreto. Si una fase se atrasa más de 1 semana, recorta scope antes de extender plazos.

Fase 0 — Fundamentos & setup

Semana 1 · ~6h

"Tener entorno listo y entender qué voy a construir."

- ☐ Instalar Python 3.11+ y VS Code
- ☐ Crear cuenta GitHub · crear repo `lego-resale-intelligence` público
- ☐ Crear cuenta en proveedor LLM (Anthropic o OpenAI) y conseguir API key
- ☐ Tutorial mínimo: "Python en 2 horas" (Corey Schafer en YouTube)
- ☐ Tutorial mínimo: "Streamlit en 30 minutos" (docs oficiales)
- ☐ Naming definitivo del proyecto (sugerencia: **BrickIntel** · **RetiredSets** · **LRI**)
- ☐ Crear documento `vision.md` en el repo con las secciones A y B de esta guía

Fase 1 — Validación manual con datos reales

Semanas 2-3 · ~14h

"Antes de construir, demostrarme a mí mismo que el modelo funciona."

- ☐ Compilar a mano lista de 30-50 sets retirados top en una hoja de cálculo
- ☐ Para cada uno: precio retail, año retiro, popularidad subjetiva 1-10, rango actual de mercado (consultar Brickset/eBay sold)
- ☐ Capturar manualmente 50 listings reales (URL + precio + descripción) en otra hoja
- ☐ Aplicar la fórmula de score a mano (en Excel/Sheets) sobre esos 50 listings
- ☐ Comprobar: ¿el ranking que sale tiene sentido para mí como humano? Si no, ajustar pesos
- ☐ Convertir las hojas en CSVs limpios → `data/catalog.csv` y `data/listings_seed.csv`

POR QUÉ ESTA FASE ES CRÍTICA

Si la lógica de scoring no funciona en Excel con datos reales, no va a funcionar en código. Ahorra 4 semanas de desarrollar algo inútil.

Fase 2 — Prototipo funcional (núcleo)

Semanas 4-7 · ~28h

"App que toma una URL y devuelve un score."

- ☐ Setup proyecto: venv + requirements.txt + estructura de carpetas
- ☐ Módulo `capture.py` : parser HTML básico (ej. eBay con BeautifulSoup)
- ☐ Módulo `extract.py` : prompt al LLM con schema Pydantic
- ☐ Módulo `identify.py` : matching set_id contra catalog.csv
- ☐ Módulo `pricing.py` + `scoring.py` : las fórmulas en código
- ☐ BBDD SQLite con SQLAlchemy · CRUD básico
- ☐ App Streamlit minimal: input URL + tabla con últimos analizados
- ☐ Test manual end-to-end con 10 URLs reales

Fase 3 — UI premium & pulido

Semanas 8–10 · ~21h

"De prototipo funcional a producto que parece producto."

- ☐ Implementar las 5 vistas (dashboard, oportunidades, detalle, comparables, métricas)
- ☐ CSS custom inyectado · paleta · tipografía Inter
- ☐ Componentes: KPI card, score badge, margin pill, condition chip
- ☐ Gráficos Plotly: distribución de scores, funnel, comparativa
- ☐ Estados: loading, empty, error
- ☐ Logo simple (Figma o text-only) y favicon
- ☐ Datos seed para demo: 30+ listings ya analizados

Fase 4 — Documentación & presentación

Semanas 11–12 · ~14h

"Convertir el proyecto en pieza de portfolio enseñable."

- ☐ README impecable: problema, solución, stack, arquitectura, capturas, demo, limitaciones
- ☐ Diagrama de arquitectura (Excalidraw o Figma)
- ☐ Vídeo demo de 2 minutos (Loom): problema → producto → resultado
- ☐ 10 capturas pulidas en `docs/screenshots/`
- ☐ Case study PDF (1–2 páginas) con narrativa de producto
- ☐ Deploy en Streamlit Community Cloud (URL pública)
- ☐ Post LinkedIn con narrativa + capturas
- ☐ 3 bullets para CV

⚠ TRAMPAS DE TIMELINE FRECUENTES

- **Saltarse Fase 1:** ir directo a programar sin validación manual. Resultado: 6 semanas de código que no aporta valor.
- **Sobre-ingeniería en Fase 2:** querer arquitectura "perfecta" antes de tener algo funcionando. Resultado: nunca llegas a Fase 3.
- **Saltarse Fase 4:** "ya documentaré después". Sin docs, el proyecto no existe para reclutadores.

Plan de construcción con Codex

Cómo usar Codex como copiloto sin convertirte en alguien que pega código que no entiende. Esta sección es la diferencia entre un proyecto defendible y un proyecto fantasma.

Filosofía: Codex es un colega senior, no un subcontratista

REGLA DE ORO

Si no entiendes lo que Codex te ha generado, no lo pegas. Le pides que te lo explique línea por línea. Si después de la explicación sigues sin entenderlo, pides una versión más simple. Repite hasta entender. Esta es la única forma de aprender mientras construyes.

Workflow recomendado por tarea

1. **Diseña en lenguaje natural primero.** Escribe en un comentario qué quieres hacer y por qué.
2. **Pídele a Codex la implementación.** Con prompt específico (ver plantillas abajo).
3. **Lee el código.** Cada línea. Sin saltarte nada.
4. **Pregunta lo que no entiendas.** "Explícame qué hace este decorator", "por qué usas dict comprehension aquí".
5. **Pega y prueba.** Ejecuta. Si rompe, copia el error literal a Codex.
6. **Refactoriza con Codex.** "Hazlo más legible", "añade type hints", "añade docstrings".
7. **Commitea con mensaje claro.** Cada commit cuenta una historia.

Cómo dividir el proyecto en bloques para Codex

Codex trabaja mejor con tareas atómicas. Un mal prompt: *"hazme la app"*. Un buen prompt es de 10–30 líneas, con contexto, restricciones y formato de salida.

Plantillas de prompts (copia y adapta)

PROMPT 1 · SETUP INICIAL

Soy un desarrollador junior que parte de cero código. Voy a construir una app Streamlit que analiza listings de LEGO en marketplaces.

Necesito que me generes:

1. El archivo requirements.txt con las dependencias mínimas

2. La estructura de carpetas exacta (lista de archivos vacíos a crear)
3. Un script `setup.sh` que cree el `venv` e instale todo
4. Un `.gitignore` apropiado para proyecto Python con `venv` y `.env`

Restricciones:

- Stack: Python 3.11, Streamlit, SQLAlchemy, SQLite, Pydantic, BeautifulSoup4, requests, Plotly, anthropic (o openai)
- Sin Docker, sin FastAPI, sin frameworks adicionales
- Comentarios en español

Devuélvelo todo como bloques de código separados, listos para pegar.

PROMPT 2 · MODELO DE DATOS

Voy a crear el modelo de datos del proyecto. Te paso el esquema en lenguaje natural y necesito que me lo conviertas a:

1. Modelos SQLAlchemy 2.0 con tipado moderno
2. Modelos Pydantic v2 equivalentes para validación
3. Un script `init_db.py` que cree las tablas

Esquema:

[PEGA AQUÍ LA SECCIÓN F COMPLETA]

Restricciones:

- Usa enums donde tenga sentido (`condition`, `status`, `action`)
- Relaciones explícitas entre tablas con foreign keys
- Cada modelo con docstring corto en español explicando qué representa
- Usa tipo ``Mapped[...]'`` de SQLAlchemy 2.0
- Pydantic con ``model_config = ConfigDict(from_attributes=True)``

Devuélveme un único archivo ``src/models.py`` y otro ``src/db.py`` con la inicialización. Explícame al final, en 5 líneas, qué decisiones de diseño has tomado.

PROMPT 3 · EXTRACCIÓN CON LLM

Necesito un módulo que tome el texto crudo de un listing y devuelva un objeto Pydantic con campos estructurados.

Schema de salida (Pydantic):

- `set_id`: str | None (4-6 dígitos)
- `title_clean`: str
- `condition`: enum (SEALED, USED_COMPLETE, USED_INCOMPLETE, UNKNOWN)
- `asking_price_eur`: float
- `shipping_eur`: float | None
- `description_summary`: str (max 200 chars)
- `risk_flags`: list[str] (de un set predefinido)

Necesito:

1. Función ``extract_listing(raw_text: str) -> ListingExtraction``

2. Llamada a la API del LLM con un system prompt claro y few-shot examples
3. Manejo de errores: timeout, JSON malformado, campos faltantes
4. Retries con backoff exponencial (max 3 intentos)
5. Test unitario con 3 ejemplos de listings reales (te los paso después)

Restricciones:

- Usa structured outputs / tool use del proveedor para forzar el schema
- Logs informativos pero sin imprimir el texto completo
- Coste por llamada estimado y loggeado

Genera el módulo `src/extract.py` completo y explícame el system prompt línea por línea.

PROMPT 4 · LÓGICA DE SCORING

Implementa el módulo de scoring siguiendo esta especificación EXACTA:

[PEGA AQUÍ LA SECCIÓN G COMPLETA]

Necesito:

1. Función `compute\_fair\_price(set\_id, condition) -> tuple[float, float, float]` devuelve (min, avg, max)
2. Función `compute\_margins(asking, shipping, fair\_price) -> Margins` donde Margins es un Pydantic con gross_eur, gross_pct, net_eur, net_pct
3. Función `compute\_score(margin\_pct, popularity, condition, risk\_flags) -> int` con los pesos exactos del documento
4. Función `categorize(score, net\_margin\_pct) -> Literal["GREEN", "YELLOW", "RED"]`

Restricciones:

- Constantes (FEES_RATE, SHIPPING_OUT, etc.) en un objeto Pydantic Settings
- Type hints estrictos
- Docstring en cada función con un ejemplo de uso
- Tests unitarios en tests/test_scoring.py cubriendo:
 - margen alto + sealed + popular → score >85
 - margen 0 + unknown → score <30
 - vetos por risk_flags críticos

Devuélveme src/scoring.py y tests/test_scoring.py. Explícame al final cómo adaptarías los pesos si quisiera priorizar liquidez sobre margen.

PROMPT 5 · VISTA STREAMLIT CON CSS PREMIUM

Voy a construir la pantalla de Dashboard del proyecto. Quiero que tenga aspecto de producto premium tipo Linear/Stripe Dashboard, no aspecto de demo de Streamlit.

Layout:

- Header con logo "LRI" + indicador de modo
- 4 KPI cards en fila: total listings, oportunidades, margen total, win rate
- Tabla "Top 10 oportunidades" con columnas: imagen, título, set_id, condición (chip), score (badge), margen (pill), CTA

- Gráfico Plotly debajo: histograma de scores

Paleta:

- Background #0a0e27 (dark) o #f8fafc (light) – quiero el toggle
- Acento #fbbf24 (amarillo LEGO)
- Verde #10b981, Rojo #ef4444

Tipografía:

- Inter para todo
- tabular-nums para números

Necesito:

1. Archivo app/pages/1_Dashboard.py
2. CSS inyectado vía `st.markdown(unsafe_allow_html=True)`
3. Componentes reutilizables (`kpi_card`, `score_badge`, `margin_pill`) en `app/components/ui.py` como funciones que devuelven HTML
4. Datos de prueba si la BBDD está vacía (mock data realista)

Restricciones:

- Sin librerías de componentes externas (no `streamlit-elements` ni similar)
- HTML/CSS inline mínimo, idealmente CSS en una variable global
- Gráfico Plotly con colores de la paleta y fondo transparente

Explícame al final cómo extraer ese CSS a un archivo `.css` separado y cargarlo correctamente desde Streamlit.

Reglas para evitar errores con Codex

Riesgo	Mitigación
Codex inventa librerías o funciones que no existen	Verifica con <code>pip show <libreria></code> antes de instalar. Lee la doc oficial 30 segundos.
Versión incompatible (ej. Pydantic v1 vs v2)	Especifica versiones exactas en el prompt y en <code>requirements.txt</code>
Código que funciona "por casualidad" pero no entiendes	Pregúntale: "explícame línea por línea". Si no convence, pídele alternativa más simple.
Acumulas 500 líneas sin probar nada	Regla: prueba cada función según se genera. <i>Test as you go</i> .
Codex sugiere "best practices" innecesarias para v1	Restricción explícita en el prompt: "v1 mínima, sin abstracciones prematuras"
Tokens consumidos con mensajes largos	Usa proyectos/conversaciones por bloque (extracción ≠ UI ≠ scoring)

Cómo validar lo que Codex genera

1. **¿Compila / corre?** Test mínimo: importar el módulo sin errores.
2. **¿Devuelve lo esperado en un caso simple?** Test happy path con 1 ejemplo.
3. **¿Maneja el caso vacío / error?** Test edge: input vacío, input malformado.
4. **¿Lo entiendo si lo leo en voz alta?** Si no, refactoriza.
5. **¿Lo podría escribir yo en una entrevista en pizarra?** Si no es código tuyo, no es código tuyo.

★ PRÁCTICA QUE PAGA 10X

Cada viernes, dedica 30 minutos a "explicarte el proyecto a ti mismo en voz alta". Abre el repo, recorre los archivos, explica qué hace cada uno como si se lo contaras a un entrevistador. Las partes donde te trabes son exactamente las que tienes que estudiar más. Esto te prepara para cualquier conversación técnica.

Entregables finales

Un proyecto sin paquete de presentación es un proyecto a medias. Estos son los 8 entregables que convierten código en pieza de portfolio.

#	Entregable	Qué incluir	Dónde
1	App funcional	Streamlit corriendo end-to-end con datos seed reales	Streamlit Community Cloud (URL pública)
2	Repo público	Código limpio, commits con historia clara, branches si procede	GitHub
3	README.md	Hero · problema · solución · stack · arquitectura · screenshots · how to run · limitaciones · roadmap	Raíz del repo
4	Diagrama de arquitectura	PNG/SVG limpio en Excalidraw o Figma · estilo similar al del bloque E	<code>docs/architecture.png</code> + README
5	Vídeo demo (2 min)	Problema (15s) → producto en uso (90s) → resultado/insights (15s)	Loom link en README
6	Capturas pulidas (8–10)	Cada vista, retina, framing limpio, sin datos personales	<code>docs/screenshots/</code>
7	Case study (PDF)	1–2 páginas: contexto · problema · solución · decisiones de producto · resultados · aprendizajes	<code>docs/case_study.pdf</code>
8	Post LinkedIn	Narrativa de 1ª persona · 3–5 capturas · qué aprendí · link al repo	LinkedIn

Plantilla de README (estructura)

```
# LEGO Resale Intelligence

> One-liner del producto · 1 frase potente

[[Demo](url-screenshot)](url-loom)
```

```
## El problema
[2-3 párrafos cortos]

## La solución
[2-3 párrafos · qué hace y para quién]

## Demo
- 🧑 [Vídeo 2min](loom)
- 🌐 [App en vivo](streamlit-cloud-url)

## Arquitectura
![arch](docs/architecture.png)

## Stack
- Python 3.11
- Streamlit, SQLAlchemy, SQLite
- Pydantic v2, BeautifulSoup4, Plotly
- Anthropic / OpenAI API

## Capturas
[3-4 imágenes destacadas]

## Cómo ejecutar localmente
```bash
git clone ...
cd lego-resale-intelligence
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # añadir API key
streamlit run app/main.py
```

## Decisiones de diseño
- Por qué Streamlit y no Next.js
- Por qué SQLite y no Postgres
- Por qué tabla de comparables manual y no scraping

## Limitaciones conocidas
- Single user, single marketplace
- Comparables no actualizados automáticamente
- ...

## Roadmap post-v1
- [ ] Multi-marketplace
- [ ] Vision LLM sobre fotos
- [ ] ...

## Licencia
MIT
```

Cómo contar el proyecto

El proyecto vale por lo que construyes y por cómo lo cuentas. La narrativa importa tanto como el código. Aquí van plantillas afinadas para cada canal.

En el CV — 3 bullets

PLANTILLA DE BULLETS

Bullet 1 (producto): Diseñé y construí *LEGO Resale Intelligence*, herramienta de e-commerce intelligence que analiza listings, estima márgenes netos y prioriza oportunidades de compra de sets descatalogados.

Bullet 2 (técnico): Implementé pipeline end-to-end (Python · Streamlit · SQLite · LLM API) con extracción estructurada vía Pydantic, scoring ponderado y dashboard analítico.

Bullet 3 (impacto): Reducción del tiempo de evaluación de listing de ~15 min (manual) a <15 s (automatizado), validado sobre 150+ casos reales con tasa de acierto del XX% sobre decisiones manuales.

En LinkedIn — post de lanzamiento

ESTRUCTURA RECOMENDADA (300–400 PALABRAS)

1. **Hook (1 línea):** "Cuesta más decidir si vale la pena un LEGO descatalogado que ensamblarlo. Lo solucioné."
2. **Contexto (3 líneas):** el problema real, lo que descubrí investigando.
3. **Producto (4 líneas):** qué hace, ejemplo concreto.
4. **Stack & decisiones (3 líneas):** 2 decisiones de producto que destacan tu criterio.
5. **Aprendizajes (3 líneas):** lo más honesto que puedas — qué te sorprendió, qué cambiarías.
6. **Demo (1 línea + link):** "App: [link] · Repo: [link] · Vídeo: [link]"
7. **Capturas:** 3–5 imágenes pulidas.

En entrevista — narrativa de 90 segundos

Cuando te pregunten "cuéntame un proyecto del que estés orgulloso":

★ STORY ARC EN 90S

0–15s • Setup: "Quería construir algo que combinara producto, datos y un dominio real. Elegí LEGO descatalogado porque..."

15–35s • Decisión clave: "Lo más interesante fue decidir qué NO hacer. Podría haber escrapeado masivamente, pero opté por X porque Y."

35–65s • Cómo lo construí: "Empecé validando manualmente con 50 listings reales antes de programar nada. Ese trabajo de Excel ahorró 4 semanas de código mal orientado. Después monté pipeline modular con..."

65–80s • Resultado & aprendizaje honesto: "El score predice X con un Y de acierto. Lo que más me sorprendió es Z. Si lo rehiciera, cambiaría W."

80–90s • Cierre + invitación a profundizar: "Está deployado en [URL]. ¿Quieres que entremos en alguna parte concreta?"

En conversación con empresa — adaptar al rol

| Rol al que aplicas | Qué destacar |
|---------------------------|---|
| PM / Producto | Decisiones de scope, qué dejaste fuera y por qué, cómo validaste demanda, métricas de éxito |
| Data / Analytics | Modelo de scoring, validación con datos reales, métricas de calidad del modelo, decisiones sobre datos |
| Junior dev | Arquitectura modular, testing, decisiones técnicas (por qué SQLite, por qué Streamlit), uso responsable de IA |
| Generalista / consultoría | End-to-end ownership, narrativa de problema → solución, comunicación visual |

⚠ FRASES QUE EVITAR AL CONTARLO

- "Lo hice todo yo solo" — sí, pero usaste IA, dilo. La transparencia transmite madurez.
- "Es un MVP" — palabra desgastada. Di "v1 enfocada" o "primera versión".
- "Voy a lanzarlo como startup" — si no es verdad, no lo digas. Si es verdad, dilo solo si tienes plan.
- "Es solo un proyecto personal" — diminutivo que destruye el valor. Dilo con peso.

Riesgos y errores a evitar

Los proyectos de portfolio fracasan más por errores de juicio que por errores técnicos. Esta sección es la lista negra de cosas que harían que tu proyecto pierda fuerza frente a una empresa exigente.

Errores de producto

| Error | Por qué duele | Antídoto |
|---|---|--|
| Querer multi-marketplace en v1 | Triplica esfuerzo, divide foco | Una fuente. Bien hecha. Punto. |
| "Algoritmo propietario" sin transparencia | Cero defensibilidad técnica | Pesos visibles, fórmulas explicables |
| Saltarse la validación manual (Fase 1) | Construyes sobre supuestos no verificados | 2 semanas de Excel antes de código |
| Definir métricas de éxito vagas | "Funciona" no es métrica | "% acierto vs decisión manual", "tiempo ahorrado", "margen real vs estimado" |

Errores técnicos

| Error | Por qué duele | Antídoto |
|--|---|--|
| Pegar 500 líneas de Codex sin entender | Te destruye en cualquier pregunta técnica | "Explícame línea por línea" antes de pegar |
| No usar Git desde el primer día | Pierdes el historial de tu pensamiento | <code>git init</code> en la primera hora |
| Commits con mensajes "fix" / "update" | Hace el repo ilegible | Conventional commits: <code>feat:</code> / <code>fix:</code> / <code>docs:</code> / <code>refactor:</code> |
| Hardcodear API keys | Te las baneará GitHub en 5 minutos | <code>.env</code> + <code>.gitignore</code> · <code>.env.example</code> sin valores |

| Error | Por qué duele | Antídoto |
|---|--|--|
| Sin tests, sin manera de detectar regresiones | Tras refactor, todo se rompe en silencio | Pytest desde la semana 4 · al menos los módulos críticos |
| Scraping agresivo sin rate-limit | IP baneada · ToS violados | Captura semi-manual o API oficial |

Errores visuales

| Error | Por qué duele | Antídoto |
|--|-------------------------------|---|
| Streamlit "out of the box" | Se ve a la legua que es demo | CSS custom · paleta · tipografía Inter |
| Demasiados colores | Aparenta amateur | 5 colores máximo · acento solo para 1–2 elementos |
| Capturas con datos personales / mock obvio | Distrae · poca seriedad | Datos seed realistas · capturas curadas |
| Logos en Comic Sans / emojis decorativos | Choque inmediato con seriedad | Sin logo o text-only en Inter Bold |
| Inconsistencia entre vistas | Sensación de prototipo | Componentes reutilizables · design tokens |

Errores de portfolio

| Error | Por qué duele | Antídoto |
|----------------------------------|-------------------------------------|---|
| README sin demo | 50% de reclutadores no clonan repos | Vídeo + capturas + URL pública SIEMPRE |
| "Trabajo en progreso" indefinido | Da sensación de inacabado | "v1 enfocada" + roadmap explícito post-v1 |
| No mencionar limitaciones | Lectura: "no tienes criterio" | Sección "Limitaciones conocidas" honesta |
| Ocultar uso de IA | Si se nota, pierdes credibilidad | Sección "Cómo usé IA" — transparente y con criterio |

| Error | Por qué duele | Antídoto |
|--|-----------------------------|--|
| Demasiada profundidad técnica en la home | Pierdes a perfiles producto | Hero accesible · profundidad solo en docs/ |

Cosas que harían parecer cutre el proyecto (lista negra explícita)

- Capturas comprimidas / pixeladas
- Errores en pantalla durante la demo
- Datos mock obvios ("Set 12345 — Lorem Ipsum")
- Mensajes de error genéricos sin manejar
- Loading sin feedback visual
- Texto en spanglish
- Faltas de ortografía en cualquier sitio público
- README en una sola sección sin estructura
- Fonts default del sistema
- Sin favicon

La trampa silenciosa más peligrosa

⚠ SÍNDROME DEL PROYECTO PERPETUAMENTE "CASI LISTO"

El error más letal en proyectos de portfolio no es que estén mal hechos — es que **nunca se terminen**. Siempre hay una feature más, un bug más, un refactor más.

Antídoto: define explícitamente la línea de meta de la v1 desde el día 1. Cuando llegues a esa línea, **publicas y paras**. Anuncias en LinkedIn, lo metes en CV, lo enseñas. El roadmap post-v1 existe pero NO bloquea la publicación.

Una v1 publicada y enseñable vale infinitamente más que una v2 hipotética en tu cabeza.

Última regla

No construyas el proyecto que crees que las empresas quieren ver.
Construye el proyecto que tú podrías defender con orgullo en cualquier mesa.

Eso es lo que las empresas realmente quieren ver.